

교과목 4 : AI 활용 애플리케이션 개발

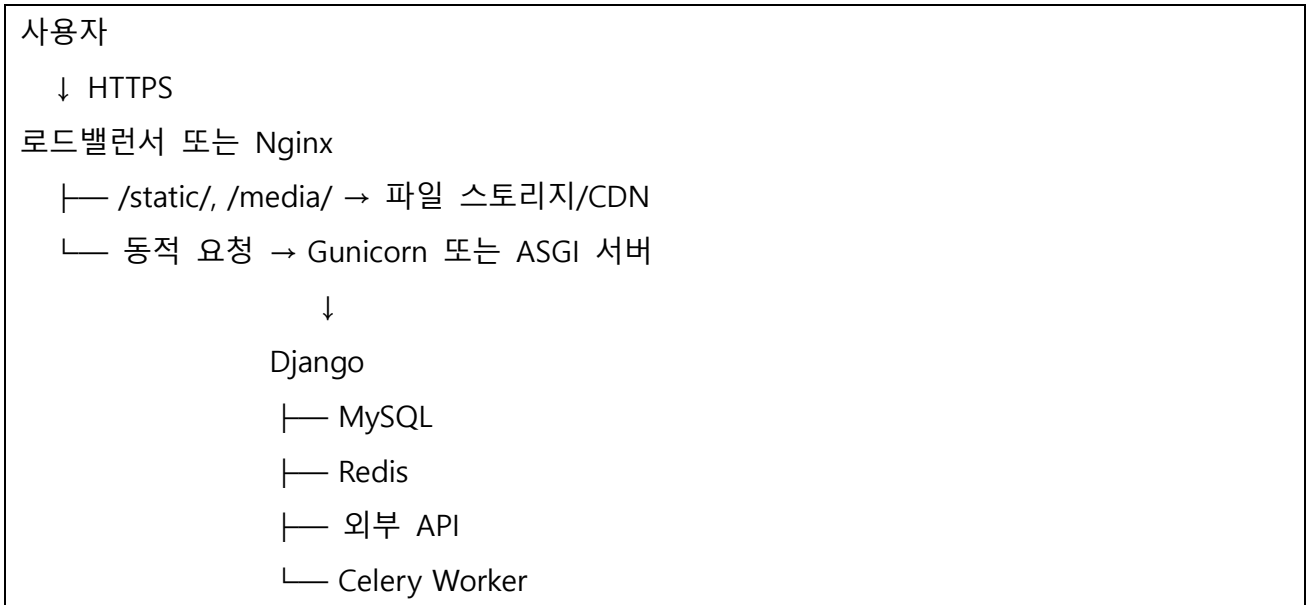
단원 2 : Django Framework

DAY3. Django 개발 환경 구축과 서비스 배포

목 차

1. 전체 운영 구조		3
2. Django 개발환경 구축		4
3. MySQL 서버 및 데이터베이스 생성		13
4. Gunicorn 배포		29
5. Nginx 구성		31
6. 정적 파일과 업로드 파일		33
7. 안전한 배포 파이프라인		34

1. 전체 운영 구조



구성요소별 역할:

- **Nginx**: HTTPS, 리버스 프록시, 정적 파일, 타임아웃 및 업로드 제한
- **Gunicorn**: Django WSGI 애플리케이션 실행
- **Uvicorn 등 ASGI 서버**: WebSocket, 장기 연결, 비동기 처리가 중요한 경우
- **MySQL InnoDB**: 트랜잭션과 관계형 데이터 저장
- **Redis**: 캐시, 세션, Celery 브로커, 분산 락
- **Celery**: 이메일, 파일 처리, 외부 API 재시도 등 비동기 작업

운영 환경에서는 `python manage.py runserver` 를 사용하지 않는다. Django 공식 배포 체크리스트도 운영용 WSGI 또는 ASGI 서버 사용을 요구한다. [Django 배포 체크리스트](#)

2. Django 개발환경 구축

1 권장 프로젝트 구조

```
myservice/  
├─ manage.py  
├─ pyproject.toml  
├─ requirements/  
│   ├─ base.txt  
│   ├─ dev.txt  
│   └─ prod.txt  
├─ config/  
│   ├─ __init__.py  
│   ├─ urls.py  
│   ├─ wsgi.py  
│   ├─ asgi.py  
│   └─ settings/  
│       ├─ __init__.py  
│       ├─ base.py  
│       ├─ local.py  
│       └─ production.py  
├─ apps/  
│   ├─ users/  
│   └─ orders/  
├─ templates/  
├─ static/  
├─ tests/  
├─ Dockerfile  
└─ compose.yaml
```

- config: 프로젝트 공통 설정
- apps: 사용자·주문·상품 등 업무 도메인별 Django 앱
- local.py: 로컬 개발 설정
- production.py: 운영 설정
- requirements: 환경별 의존성

2. Python 가상환경

Windows PowerShell:

```
py -m venv .venv  
.venv\Scripts\Activate.ps1  
python -m pip install --upgrade pip
```

Linux/macOS:

```
python3 -m venv .venv  
source .venv/bin/activate  
python -m pip install --upgrade pip
```

3. Django 와 MySQL 드라이버 설치

```
pip install "Django~=5.2.0" mysqlclient gunicorn
```

mysqlclient 는 Django 에서 일반적으로 권장되는 MySQL 네이티브 드라이버다. 다만 OS 에 따라 C 라이브러리와 컴파일 도구가 필요하다.

Ubuntu/Debian 예:

```
sudo apt-get update  
sudo apt-get install ₩  
python3-dev ₩  
default-libmysqlclient-dev ₩  
build-essential ₩  
pkg-config
```

그다음:

```
pip install mysqlclient
```

Windows 에서는 wheel 설치 여부나 Visual C++ 빌드 도구가 문제가 될 수 있다. 팀 전체의 환경 일관성이 중요하다면 개발 DB 까지 Docker Compose 로 실행하는 것이 편하다.

프로젝트 생성:

```
django-admin startproject config .  
python manage.py startapp users  
python manage.py check  
python manage.py runserver
```

4, 의존성 고정

간단한 방식:

```
pip freeze > requirements/prod.txt
```

실무에서는 pip-tools, Poetry, uv 등을 사용해 직접 의존성과 하위 의존성을 함께 잠그는 방식이 좋다.

CI, 개발자 PC, 운영 서버가 동일한 버전을 설치해야 한다. 단순히 다음처럼 범위만 지정하면 설치 시점에 따라 결과가 달라질 수 있다.

```
Django>=5  
mysqlclient
```

운영 재현성을 위해 최종 설치 버전을 잠가야 한다.

5. 환경변수 관리

개발용 .env 예:

```
DJANGO_SETTINGS_MODULE=config.settings.local  
DJANGO_SECRET_KEY=local-only-secret  
DJANGO_DEBUG=true  
  
DB_NAME=myservice  
DB_USER=myservice_app  
DB_PASSWORD=local-password  
DB_HOST=127.0.0.1  
DB_PORT=3306
```

```
.gitignore:  
.venv/
```

```
.env
__pycache__/
*.py[cod]
staticfiles/
media/
.pytest_cache/
.coverage
```

운영 비밀값은 다음 중 하나에 저장한다.

- AWS Secrets Manager
- GCP Secret Manager
- Azure Key Vault
- Kubernetes Secret
- systemd 의 권한 제한된 환경 파일
- CI/CD 플랫폼의 encrypted secret

소스코드, Dockerfile, Docker 이미지, Git 저장소에 운영 비밀번호를 포함하면 안 된다.

6. 환경별 Django 설정

6.1 공통 설정

config/settings/base.py:

```
import os
from pathlib import Path

BASE_DIR = Path(__file__).resolve().parents[2]

SECRET_KEY = os.environ["DJANGO_SECRET_KEY"]

INSTALLED_APPS = [
    "django.contrib.admin",
    "django.contrib.auth",
    "django.contrib.contenttypes",
    "django.contrib.sessions",
```

```

    "django.contrib.messages",
    "django.contrib.staticfiles",
    "apps.users",
    "apps.orders",
]

MIDDLEWARE = [
    "django.middleware.security.SecurityMiddleware",
    "django.contrib.sessions.middleware.SessionMiddleware",
    "django.middleware.common.CommonMiddleware",
    "django.middleware.csrf.CsrfViewMiddleware",
    "django.contrib.auth.middleware.AuthenticationMiddleware",
    "django.contrib.messages.middleware.MessageMiddleware",
    "django.middleware.clickjacking.XFrameOptionsMiddleware",
]

ROOT_URLCONF = "config.urls"
WSGI_APPLICATION = "config.wsgi.application"
ASGI_APPLICATION = "config.asgi.application"

LANGUAGE_CODE = "ko-kr"
TIME_ZONE = "Asia/Seoul"

USE_I18N = True
USE_TZ = True

STATIC_URL = "/static/"
STATIC_ROOT = BASE_DIR / "staticfiles"

MEDIA_URL = "/media/"
MEDIA_ROOT = BASE_DIR / "media"

DEFAULT_AUTO_FIELD = "django.db.models.BigAutoField"

```

SECRET_KEY 는 다음처럼 위험한 기본값을 두지 않는 것이 좋다.

권장하지 않음

```
SECRET_KEY = os.getenv("DJANGO_SECRET_KEY", "temporary-secret")
```

운영 환경변수가 누락되어도 애플리케이션이 실행되기 때문이다. os.environ[...]로 즉시 실패시키는 편이 안전하다.

6.2 개발환경 MySQL 설정

config/settings/local.py:

```
from .base import *

DEBUG = True

ALLOWED_HOSTS = [
    "localhost",
    "127.0.0.1",
]

DATABASES = {
    "default": {
        "ENGINE": "django.db.backends.mysql",
        "NAME": os.environ["DB_NAME"],
        "USER": os.environ["DB_USER"],
        "PASSWORD": os.environ["DB_PASSWORD"],
        "HOST": os.environ.get("DB_HOST", "127.0.0.1"),
        "PORT": os.environ.get("DB_PORT", "3306"),
        "OPTIONS": {
            "charset": "utf8mb4",
            "init_command": (
                "SET sql_mode="
                "'STRICT_TRANS_TABLES,NO_ENGINE_SUBSTITUTION'"
            ),
        },
    },
}
```

실행:

```
python manage.py runserver ₩
--settings=config.settings.local
```

또는 환경변수 지정:

```
export DJANGO_SETTINGS_MODULE=config.settings.local
python manage.py runserver
```

PowerShell:

```
$env:DJANGO_SETTINGS_MODULE = "config.settings.local"
python manage.py runserver
```

6.3 운영환경 MySQL 설정

config/settings/production.py:

```
from .base import *

DEBUG = False

ALLOWED_HOSTS = [
    "api.example.com",
]

CSRF_TRUSTED_ORIGINS = [
    "https://api.example.com",
]

DATABASES = {
    "default": {
        "ENGINE": "django.db.backends.mysql",
        "NAME": os.environ["DB_NAME"],
        "USER": os.environ["DB_USER"],
        "PASSWORD": os.environ["DB_PASSWORD"],
        "HOST": os.environ["DB_HOST"],
```

```

"PORT": os.environ.get("DB_PORT", "3306"),
"CONN_MAX_AGE": 60,
"CONN_HEALTH_CHECKS": True,
"OPTIONS": {
    "charset": "utf8mb4",
    "isolation_level": "read committed",
    "init_command": (
        "SET sql_mode="
        "'STRICT_TRANS_TABLES,NO_ENGINE_SUBSTITUTION'"
    ),
    # 관리형 DB 가 TLS 를 요구하는 경우
    # "ssl": {
    #     "ca": "/etc/ssl/certs/mysql-ca.pem",
    # },
},
}

}

SECURE_SSL_REDIRECT = True
SESSION_COOKIE_SECURE = True
CSRF_COOKIE_SECURE = True

SECURE_HSTS_SECONDS = 31536000
SECURE_HSTS_INCLUDE_SUBDOMAINS = True
SECURE_HSTS_PRELOAD = True

SECURE_CONTENT_TYPE_NOSNIFF = True
X_FRAME_OPTIONS = "DENY"

# 신뢰할 수 있는 프록시에서 HTTPS 가 종료되는 경우에만 설정
SECURE_PROXY_SSL_HEADER = ("HTTP_X_FORWARDED_PROTO", "https")

```

MySQL 서버가 `sql_mode` 를 중앙에서 엄격하게 관리하고 있다면 `init_command` 를 Django 설정에 중복해서 넣지 않아도 된다. 중요한 것은 실제 연결의 SQL mode 가 기대값과 일치하는지 확인하는 것이다.

```
SELECT @@SESSION.sql_mode;  
SELECT @@SESSION.transaction_isolation;
```

3. MySQL 서버 및 데이터베이스 생성

1. 데이터베이스 생성

MySQL 관리자 계정으로 접속한다.

```
mysql -u root -p
```

데이터베이스 생성:

```
CREATE DATABASE myservice  
    CHARACTER SET utf8mb4  
    COLLATE utf8mb4_0900_ai_ci;
```

애플리케이션 계정 생성:

```
CREATE USER 'myservice_app'@'10.%'  
    IDENTIFIED BY 'strong-password';
```

권한 부여:

```
GRANT SELECT, INSERT, UPDATE, DELETE,  
    CREATE, ALTER, INDEX, REFERENCES  
ON myservice.*  
TO 'myservice_app'@'10.%';  
  
FLUSH PRIVILEGES;
```

호스트 부분은 환경에 맞게 제한한다.

'myservice_app'@'%' 는 어느 호스트에서든 접속할 수 있게 허용하므로, 보안그룹이나 방화벽이 별도로 강제되지 않는다면 피하는 것이 좋다.

운영에서는 다음 계정을 분리하는 것이 더 안전하다.

- 런타임 계정: SELECT, INSERT, UPDATE, DELETE
- 마이그레이션 계정: CREATE, ALTER, INDEX, DROP
- 백업 계정: 백업에 필요한 읽기 권한
- 관리자 계정: DBA 작업 전용

Django 가 시작될 때마다 스키마를 변경하지 않도록, 일반 웹 프로세스는 제한된 계정으로 실행하고 CI/CD 의 마이그레이션 단계만 DDL 권한 계정을 사용하는 방식도 가능하다.

2 문자셋

MySQL 에서는 utf8 보다 utf8mb4 를 사용한다.

```
SHOW VARIABLES LIKE 'character_set%';  
SHOW VARIABLES LIKE 'collation%';
```

확인 대상:

```
character_set_server  
character_set_database  
character_set_connection  
collation_server  
collation_database
```

MySQL 의 utf8 은 역사적으로 3 바이트 UTF-8 을 의미하며 모든 유니코드 문자를 저장하지 못할 수 있다. utf8mb4 는 4 바이트 유니코드를 지원하므로 한글뿐 아니라 이모지와 보조 평면 문자까지 저장할 수 있다. MySQL 공식 문서도 신규 애플리케이션에 utf8mb4 사용을 권장한다.

[MySQL 문자셋 문서](#)

콜레이션은 검색 및 정렬 의미에 영향을 준다.

- 대소문자 구분 여부
- 악센트 구분 여부
- 문자열의 고유성 판단
- 정렬 순서

예를 들어 대소문자를 무시하는 콜레이션에서는 User 와 user 가 같은 고유값으로 취급될 수 있다. 사용자명.이메일.코드 값의 비교 규칙을 설계 단계에서 정해야 한다.

3 저장 엔진

트랜잭션, 외래키, 행 수준 잠금을 사용하려면 InnoDB 를 사용한다.

```
SHOW VARIABLES LIKE 'default_storage_engine';
```

결과가 다음이어야 한다.

```
InnoDB
```

테이블별 확인:

```
SELECT
    TABLE_NAME,
    ENGINE
FROM information_schema.TABLES
WHERE TABLE_SCHEMA = 'myservice';
```

레거시 환경에 MyISAM 테이블이 있다면 트랜잭션과 외래키가 기대대로 작동하지 않을 수 있으므로 반드시 점검해야 한다.

4. MySQL 타임존 구성

Django 에서 일반적으로 다음을 사용한다.

```
USE_TZ = True
TIME_ZONE = "Asia/Seoul"
```

Django 는 애플리케이션 내부에서 timezone-aware datetime 을 사용하는 방향이 안전하다. 서버와 DB 는 가급적 UTC 기준으로 운영하고 표시 단계에서 한국 시간으로 변환한다. MySQL 타임존 테이블이 비어 있으면 일부 날짜·시간 변환 기능이 정상적으로 작동하지 않을 수 있다.

Linux MySQL 서버 예:

```
mysql_tzinfo_to_sql /usr/share/zoneinfo ₩
| mysql -u root -p mysql
```

확인:

```
SELECT
    @@GLOBAL.time_zone,
    @@SESSION.time_zone;
```

```
SELECT CONVERT_TZ(  
    '2026-08-12 00:00:00',  
    'UTC',  
    'Asia/Seoul'  
);
```

CONVERT_TZ() 결과가 NULL 이면 타임존 정보가 적재되지 않았을 가능성이 있다.

5. 데이터베이스 연결 검증

Django 설정 검사:

```
python manage.py check --database default
```

마이그레이션:

```
python manage.py migrate
```

MySQL 셸:

```
python manage.py dbshell
```

Django 셸에서 연결 확인:

```
python manage.py shell  
from django.db import connection  
  
with connection.cursor() as cursor:  
    cursor.execute("""  
        SELECT  
            VERSION(),  
            @@SESSION.sql_mode,  
            @@SESSION.transaction_isolation  
    """)  
    print(cursor.fetchone())
```

문자셋 확인:

```
with connection.cursor() as cursor:
    cursor.execute("""
        SELECT
            @@character_set_client,
            @@character_set_connection,
            @@character_set_results
    """)
    print(cursor.fetchone())
```

6. 모델과 마이그레이션

예제 모델:

```
from django.db import models

class Order(models.Model):
    class Status(models.TextChoices):
        PENDING = "pending", "대기"
        PAID = "paid", "결제 완료"
        CANCELED = "canceled", "취소"

    customer_id = models.BigIntegerField(db_index=True)

    status = models.CharField(
        max_length=20,
        choices=Status.choices,
        default=Status.PENDING,
        db_index=True,
    )

    amount = models.DecimalField(
        max_digits=12,
```

```

        decimal_places=2,
    )

    idempotency_key = models.CharField(
        max_length=64,
        unique=True,
    )

    created_at = models.DateTimeField(
        auto_now_add=True,
    )

    class Meta:
        indexes = [
            models.Index(
                fields=["customer_id", "-created_at"],
                name="order_customer_created_idx",
            ),
        ]
        constraints = [
            models.CheckConstraint(
                condition=models.Q(amount__gte=0),
                name="order_amount_nonnegative",
            ),
        ]

```

마이그레이션 생성 및 확인:

```

python manage.py makemigrations
python manage.py sqlmigrate orders 0001
python manage.py migrate
python manage.py showmigrations

```

중요 원칙:

- 마이그레이션 파일은 Git 에 커밋한다.
- 모델 변경과 해당 마이그레이션을 같은 변경 단위로 관리한다.
- 운영 서버에서 makemigrations 를 실행하지 않는다.
- CI 에서 생성하고 검증한 마이그레이션만 운영에서 migrate 한다.
- --fake 를 일상적인 오류 회피 수단으로 사용하지 않는다.

CI 검사:

```
python manage.py makemigrations --check --dry-run
python manage.py migrate --check
```

배포 전 SQL 확인:

```
python manage.py migrate --plan
python manage.py sqlmigrate orders 0002
```

7. MySQL strict mode

MySQL strict mode 가 꺼져 있으면 잘못된 값을 오류 없이 절단하거나 변환하는 문제가 발생할 수 있다.

예:

- 너무 긴 문자열이 잘림
- 범위를 넘은 숫자가 보정됨
- 잘못된 날짜가 저장됨
- 데이터 손상이 즉시 발견되지 않음

권장 설정:

```
SET GLOBAL sql_mode =
'STRICT_TRANS_TABLES,NO_ENGINE_SUBSTITUTION';
```

운영에서는 MySQL 설정 파일에 반영한다.

```
[mysqld]
sql_mode=STRICT_TRANS_TABLES,NO_ENGINE_SUBSTITUTION
character-set-server=utf8mb4
```

```
collation-server=utf8mb4_0900_ai_ci
default-storage-engine=InnoDB
```

MySQL 버전과 기존 데이터에 따라 NO_ZERO_DATE, ERROR_FOR_DIVISION_BY_ZERO 등의 모드도 검토한다. 기존 시스템에 strict mode 를 갑자기 적용하면 이전에는 허용되던 데이터 입력이 실패할 수 있으므로 스테이징에서 검증해야 한다.

8. 트랜잭션과 격리수준

8.1 기본 트랜잭션 처리

```
from django.db import transaction
```

```
@transaction.atomic
```

```
def pay_order(order_id, payment_id):
```

```
    order = (
        Order.objects
        .select_for_update()
        .get(pk=order_id)
    )
```

```
    if order.status != Order.Status.PENDING:
        raise ValueError("이미 처리된 주문입니다.")
```

```
    Payment.objects.create(
        order=order,
        external_id=payment_id,
    )
```

```
    order.status = Order.Status.PAID
    order.save(update_fields=["status"])
```

select_for_update()는 반드시 트랜잭션 안에서 사용해야 한다.

8.2 MySQL 격리수준

InnoDB 서버의 일반적인 기본 격리수준은 REPEATABLE READ 다. 반면 Django에서는 일반적으로 READ COMMITTED 를 사용하도록 구성한다.

```
"OPTIONS": {  
    "isolation_level": "read committed",  
}
```

MySQL InnoDB 는 다음 네 격리수준을 지원한다.

- READ UNCOMMITTED
- READ COMMITTED
- REPEATABLE READ
- SERIALIZABLE

[MySQL InnoDB 격리수준 문서](#)

REPEATABLE READ 에서는 범위 조건과 잠금 읽기에 따라 gap lock 또는 next-key lock 이 발생할 수 있다. 이 때문에 예상보다 넓은 범위의 삽입이 대기하거나 교착상태가 증가할 수 있다.

격리수준을 변경하기 전에 다음을 확인해야 한다.

- 조회 일관성 요구
- phantom read 허용 여부
- gap lock 영향
- 복제의 binary log 형식
- 기존 SQL 및 배치 프로그램의 가정

8.3 교착상태

InnoDB 에서는 정상적으로 운영되는 시스템에서도 교착상태가 발생할 수 있다. 두 트랜잭션이 서로 반대 순서로 자원을 잠그면 MySQL 이 하나를 중단한다.

예방 원칙:

- 여러 행을 갱신할 때 항상 동일한 순서로 잠근다.
- 트랜잭션을 짧게 유지한다.
- 외부 API 호출을 트랜잭션 안에서 하지 않는다.
- 검색 조건에 적절한 인덱스를 둔다.
- 교착상태 오류는 제한된 횟수로 재시도한다.
- idempotency key 를 사용해 재시도 중복 처리를 막는다.

최근 교착상태 확인:

```
SHOW ENGINE INNODB STATUS;
```

잠금 관련 정보:

```
SELECT *
```

```
FROM performance_schema.data_lock_waits;
```

9. Django 개발용 MySQL 을 Docker 로 실행하기

compose.yaml:

```
services:
  db:
    image: mysql:8.4
    command:
      - --character-set-server=utf8mb4
      - --collation-server=utf8mb4_0900_ai_ci
      - --default-storage-engine=InnoDB
      - --transaction-isolation=READ-COMMITTED
    environment:
      MYSQL_DATABASE: myservice
      MYSQL_USER: myservice_app
      MYSQL_PASSWORD: local-password
      MYSQL_ROOT_PASSWORD: local-root-password
      TZ: Asia/Seoul
    ports:
      - "3306:3306"
    volumes:
      - mysql_data:/var/lib/mysql
    healthcheck:
      test:
        - CMD
        - mysqladmin
        - ping
        - -h
        - localhost
        - -uroot
```

```
- -plocal-root-password
```

```
interval: 5s
```

```
timeout: 3s
```

```
retries: 20
```

```
volumes:
```

```
mysql_data:
```

실행:

```
docker compose up -d db
```

```
docker compose ps
```

```
docker compose logs db
```

```
python manage.py migrate
```

Django 도 컨테이너 안에서 실행한다면:

```
DB_HOST=db
```

```
DB_PORT=3306
```

호스트 PC 에서 Django 를 실행하면:

```
DB_HOST=127.0.0.1
```

```
DB_PORT=3306
```

depends_on 만으로 MySQL 이 쿼리를 받을 준비가 됐다고 보장할 수 없다. health check 와 애플리케이션 시작 재시도가 필요하다.

10. 연결 수와 연결 수명

운영 설정:

```
DATABASES = {  
    "default": {  
        # ...  
        "CONN_MAX_AGE": 60,  
        "CONN_HEALTH_CHECKS": True,  
    }  
}
```

```
}
```

CONN_MAX_AGE 는 Django 가 DB 연결을 재사용하는 시간이다.

- 0: 요청 종료 시 연결 종료
- 양수: 해당 초 동안 재사용
- None: 연결을 계속 유지

무조건 크게 설정하면 좋은 것이 아니다. 총 연결 수를 계산해야 한다.

최대 DB 연결 수

≈ 웹 서버 수 × Gunicorn 워커 수

- + Celery 프로세스 수
- + 관리 명령
- + 배치 프로세스
- + 모니터링
- + 장애조치 여유분

예:

웹 서버 4 대 × 워커 5 개 = 20

Celery 프로세스 = 8

배치/관리/모니터링 = 5

여유분 = 10

예상 필요 연결 수 = 43

MySQL 서버 확인:

```
SHOW VARIABLES LIKE 'max_connections';  
SHOW STATUS LIKE 'Threads_connected';  
SHOW STATUS LIKE 'Threads_running';  
SHOW STATUS LIKE 'Max_used_connections';
```

주의할 점:

- max_connections 만 높이면 연결별 메모리 소비도 증가한다.
- Gunicorn 워커를 늘리면 DB 연결도 증가한다.
- ASGI 의 동시 요청 수와 DB 동시성은 별도로 제한해야 한다.

- 관리형 DB 는 인스턴스 크기별 최대 연결 수가 제한될 수 있다.
- 오래된 연결은 MySQL 의 wait_timeout 으로 끊길 수 있다.
- CONN_HEALTH_CHECKS=True 는 재사용 전에 연결 상태를 점검하는 데 유용하다.

대규모 서비스에서는 MySQL ProxySQL, 클라우드 DB Proxy 등을 검토할 수 있지만, 애플리케이션과 프록시 양쪽에서 풀 크기를 과도하게 잡으면 오히려 연결 폭증이 발생한다.

11. 쿼리 성능과 인덱스

11.1 N+1 문제

나쁜 예:

```
orders = Order.objects.all()

for order in orders:
    print(order.customer.name)
```

주문 수만큼 customer 쿼리가 추가될 수 있다.

개선:

```
orders = Order.objects.select_related("customer")
```

다대다 및 역방향 관계:

```
orders = Order.objects.prefetch_related("items")
```

11.2 실행계획

MySQL 에서:

```
EXPLAIN
SELECT *
FROM orders_order
WHERE customer_id = 100
ORDER BY created_at DESC
LIMIT 20;
```

실제 실행시간까지 확인:

```
EXPLAIN ANALYZE
SELECT *
FROM orders_order
WHERE customer_id = 100
ORDER BY created_at DESC
LIMIT 20;
```

EXPLAIN ANALYZE 는 쿼리를 실제 실행하므로 운영에서 UPDATE 나 DELETE 에 무심코 사용하면 안 된다. 실행계획에서는 다음을 본다.

- 전체 테이블 스캔 여부
- 사용 인덱스
- 예상 행 수와 실제 행 수 차이
- 정렬 및 임시 테이블
- 조인 순서
- 반복 횟수

[MySQL EXPLAIN 문서](#)

11.3 복합 인덱스

다음 쿼리가 많다면:

```
WHERE customer_id = ?
ORDER BY created_at DESC
```

복합 인덱스가 적합하다.

```
class Meta:
    indexes = [
        models.Index(
            fields=["customer_id", "-created_at"],
            name="order_customer_created_idx",
        ),
    ]
```

MySQL 복합 인덱스는 컬럼 순서가 중요하다.

(customer_id, created_at)

인덱스가 있을 때 일반적으로:

- customer_id = ?에 사용 가능
- customer_id = ? AND created_at > ?에 사용 가능
- created_at 만 조건으로 쓸 때는 효율적으로 사용되지 않을 수 있음

실무 원칙:

- 실제 쿼리 패턴에 맞춰 인덱스를 설계한다.
- 외래키와 자주 조회하는 조건을 확인한다.
- 인덱스를 너무 많이 만들면 쓰기와 저장공간 비용이 증가한다.
- 낮은 선택도의 Boolean 컬럼 단독 인덱스는 효과가 작을 수 있다.
- 긴 VARCHAR 인덱스는 크기와 콜레이션 비용을 검토한다.
- 대형 목록에는 offset 방식보다 keyset pagination 을 고려한다.

12. 대형 테이블 마이그레이션

MySQL에서는 ALTER TABLE 이 변경 종류와 버전에 따라 다음 방식 중 하나를 사용할 수 있다.

- INSTANT
- INPLACE
- COPY

일부 변경은 빠르게 끝나지만 일부 변경은 테이블 전체 복사, 장시간 실행 또는 메타데이터 락을 발생시킬 수 있다.

운영 배포 전에 확인한다.

```
python manage.py sqlmigrate orders 0005
python manage.py migrate --plan
```

생성된 ALTER TABLE 을 스테이징의 운영 유사 데이터로 검증한다.

MySQL Online DDL 도 작업 시작과 종료 단계에서 짧은 exclusive metadata lock 이 필요할 수 있다. 오래 실행 중인 트랜잭션이 있으면 이 잠금 획득이 지연되고, 그 뒤의 요청들이 줄줄이 대기할 수 있다. [MySQL Online DDL 문서](#)

안전한 컬럼 추가 예

새로운 필수 컬럼을 한 번에 추가하고 모든 데이터를 채우지 않는다.

1. nullable 컬럼을 추가한다.
2. 기존 코드와 신규 코드가 함께 동작하도록 배포한다.
3. 작은 배치 단위로 기존 데이터를 채운다.
4. 새 컬럼 읽기로 전환한다.
5. 누락 데이터가 있는지 검증한다.
6. NOT NULL 과 제약조건을 적용한다.
7. 호환 기간 후 기존 컬럼을 제거한다.

Expand → Backfill → Switch → Contract

수천만 행의 테이블에서는 다음 도구도 검토할 수 있다.

- gh-ost
- pt-online-schema-change
- 관리형 MySQL 의 온라인 스키마 변경 기능

이러한 도구도 복제 지연, 트리거, 외래키, 디스크 공간, 부하에 영향을 주므로 자동으로 안전한 것은 아니다.

4. Gunicorn 배포

설치:

```
pip install gunicorn
```

실행:

```
gunicorn config.wsgi:application ₩
--bind 127.0.0.1:8000 ₩
--workers 4 ₩
--timeout 30 ₩
--access-logfile - ₩
--error-logfile -
```

워커 수는 공식처럼 외운 숫자가 아니라 부하 테스트로 결정한다.

확인 대상:

- CPU 사용량
- 워커당 메모리
- p95/p99 응답시간
- MySQL 연결 수
- MySQL Threads_running
- 외부 API 대기
- 요청 타임아웃
- 오류율

systemd 설정

```
/etc/systemd/system/myservice.service:
[Unit]
Description=MyService Django
After=network.target

[Service]
User=myservice
```

```
Group=www-data
WorkingDirectory=/srv/myservice/current
EnvironmentFile=/etc/myservice/myservice.env
ExecStart=/srv/myservice/venv/bin/gunicorn \#
    config.wsgi:application \#
    --bind unix:/run/myservice/gunicorn.sock \#
    --workers 4 \#
    --timeout 30 \#
    --access-logfile - \#
    --error-logfile -
Restart=always
RestartSec=3
PrivateTmp=true

[Install]
WantedBy=multi-user.target
```

적용:

```
sudo systemctl daemon-reload
sudo systemctl enable --now myservice
sudo systemctl status myservice
sudo journalctl -u myservice -f
```

5. Nginx 구성

```
upstream myservice_backend {
    server unix:/run/myservice/gunicorn.sock;
}

server {
    listen 80;
    server_name api.example.com;

    return 301 https://$host$request_uri;
}

server {
    listen 443 ssl http2;
    server_name api.example.com;

    ssl_certificate
        /etc/letsencrypt/live/api.example.com/fullchain.pem;
    ssl_certificate_key
        /etc/letsencrypt/live/api.example.com/privkey.pem;

    client_max_body_size 10m;

    location /static/ {
        alias /srv/myservice/current/staticfiles/;
        expires 30d;
        add_header Cache-Control "public, immutable";
    }

    location /media/ {
        alias /srv/myservice/shared/media/;
    }
}
```

```
location / {  
    proxy_pass http://myservice_backend;  
  
    proxy_set_header Host $host;  
    proxy_set_header X-Real-IP $remote_addr;  
    proxy_set_header X-Forwarded-For  
        $proxy_add_x_forwarded_for;  
    proxy_set_header X-Forwarded-Proto $scheme;  
  
    proxy_connect_timeout 5s;  
    proxy_read_timeout 30s;  
}  
}
```

검증:

```
sudo nginx -t  
sudo systemctl reload nginx
```

6. 정적 파일과 업로드 파일

정적 파일 수집:

```
python manage.py collectstatic --noinput
```

구분:

- **static:** 개발자가 배포하는 CSS, JavaScript, 이미지
- **media:** 사용자가 업로드하는 영속 파일

정적 파일에는 해시 이름을 사용하는 것이 좋다.

```
STORAGES = {
    "default": {
        "BACKEND":
            "django.core.files.storage.FileSystemStorage",
    },
    "staticfiles": {
        "BACKEND": (
            "django.contrib.staticfiles.storage."
            "ManifestStaticFilesStorage"
        ),
    },
}
```

여러 서버를 운영할 때 사용자 업로드 파일을 서버 로컬 디스크에 저장하면 서버마다 파일이 달라진다. S3 와 같은 객체 스토리지에 저장하고 CDN 을 연결하는 방식이 일반적이다.

7. 안전한 배포 파이프라인

1. 안전한 배포 파이프라인

코드 푸시

- 린트 및 정적 분석
- 단위·통합 테스트
- 마이그레이션 누락 검사
- 이미지 또는 패키지 빌드
- 스테이징 배포
- MySQL DDL 검증
- 운영 마이그레이션
- 애플리케이션 롤링 배포
- 헬스체크
- 지표 및 로그 감시

CI 검사:

```
python manage.py check
python manage.py makemigrations --check --dry-run
python manage.py test
python manage.py collectstatic --noinput
python manage.py migrate --check
python manage.py check --deploy ₩
--settings=config.settings.production
```

운영 배포:

```
python manage.py migrate --noinput ₩
--settings=config.settings.production

python manage.py collectstatic --noinput ₩
--settings=config.settings.production

sudo systemctl restart myservice
```

대규모 서비스라면 단순 restart 보다 다음 방식을 사용한다.

- 인스턴스를 한 대씩 배포
- readiness 통과 후 트래픽 투입
- 이전 인스턴스의 연결 종료 대기
- 실패 시 새 버전 트래픽 제거
- DB 스키마는 이전 애플리케이션과도 호환 유지

DB 마이그레이션은 애플리케이션 롤백보다 되돌리기 어렵다. 따라서 “코드가 롤백되더라도 새 스키마에서 동작하는가”를 먼저 설계해야 한다.

2. 헬스체크

```
from django.db import connection
from django.http import JsonResponse

def liveness(request):
    return JsonResponse({"status": "ok"})

def readiness(request):
    try:
        with connection.cursor() as cursor:
            cursor.execute("SELECT 1")
            cursor.fetchone()

        return JsonResponse({"status": "ready"})

    except Exception:
        return JsonResponse(
            {"status": "not_ready"},
            status=503,
        )
```

구분:

- **Liveness:** Django 프로세스 자체가 살아 있는가
- **Readiness:** MySQL 등 필수 자원에 연결해 요청을 처리할 수 있는가

DB 가 잠시 느린 상황에서 liveness 까지 실패시키면 모든 프로세스가 동시에 재시작되어 장애가 더 커질 수 있다.

3. MySQL 백업과 시점 복구

3.1 논리 백업

InnoDB 데이터베이스 백업:

```
mysqldump ₩
--host=mysql.example.internal ₩
--user=backup_user ₩
--password ₩
--single-transaction ₩
--routines ₩
--events ₩
--triggers ₩
--set-gtid-purged=OFF ₩
myservice ₩
> myservice.sql
```

--single-transaction 은 InnoDB 에서 일관된 스냅샷을 얻으면서 일반적인 테이블 잠금을 피하는데 유용하다. MySQL 공식 문서도 InnoDB 논리 백업에 이 옵션을 설명한다. [MySQL 백업 방법](#)

복구:

```
mysql ₩
--host=mysql-restore.example.internal ₩
--user=restore_user ₩
--password ₩
myservice_restore ₩
< myservice.sql
```

주의:

- --single-transaction 백업 중 DDL 이 실행되면 일관성 문제가 생길 수 있다.
- 대형 DB 에서 mysqldump 는 백업과 복구 시간이 길다.
- 저장 프로시저, 이벤트, 트리거, 사용자 권한은 별도로 검토한다.
- 백업 파일에도 개인정보가 있으므로 암호화하고 접근을 제한한다.

3.2 물리 백업과 PITR

대형 데이터베이스는 관리형 DB 스냅샷 또는 물리 백업을 사용한다.

시점 복구에는 일반적으로 다음이 필요하다.

전체 백업 또는 스냅샷 + 이후의 MySQL binary log = 특정 시점으로 복구

MySQL 의 시점 복구는 binary log 를 기반으로 한다. [MySQL 백업·복구 문서](#)

필수 운영 항목:

- 자동 백업
- binary log 보존 기간
- 백업 암호화
- 다른 리전 또는 계정에 사본 보관
- RPO 정의
- RTO 정의
- 정기적인 복구 훈련
- 복구 후 데이터 검증

백업 성공 로그만 확인해서는 안 된다. 격리된 MySQL 인스턴스에 실제로 복구한 뒤 다음을 검증해야 한다.

- 테이블과 인덱스 존재
- 주요 테이블 레코드 수
- 문자셋과 콜레이션
- 애플리케이션 마이그레이션 상태
- Django 애플리케이션 기동
- 로그인·주문 등 핵심 기능

4. 복제와 읽기 DB

MySQL 복제본을 읽기 전용으로 사용할 수 있다.

```
DATABASES = {  
    "default": {  
        # 쓰기 DB  
    },  
    "replica": {  
        # 읽기 복제본  
    },  
}
```

하지만 단순히 읽기 요청을 모두 replica 로 보내면 안 된다.
복제 지연 때문에 다음 상황이 발생할 수 있다.

사용자가 주문 저장
→ 즉시 주문 상세 조회
→ replica 에 아직 반영되지 않음
→ "주문 없음" 응답

따라서 다음 전략이 필요하다.

- 쓰기 직후 일정 시간 primary 에서 읽기
- 정합성이 중요한 조회는 항상 primary 사용
- 통계·검색·관리 목록만 replica 사용
- replica lag 을 모니터링
- 장애조치 후 연결 갱신
- Django database router 를 명시적으로 설계

복제는 백업의 대체가 아니다. 사용자가 데이터를 삭제하면 삭제 작업도 replica 에 복제된다.

5. 관측성과 모니터링

Django 지표:

- 요청 수
- HTTP 오류율

- p50/p95/p99 응답시간
- Gunicorn 워커 타임아웃
- 배포 버전별 오류율
- Celery 큐 길이와 실패 수

MySQL 지표:

- Threads_connected
- Threads_running
- Max_used_connections
- slow query
- lock wait
- deadlock
- buffer pool hit ratio
- 임시 테이블 생성
- 디스크 사용량과 IOPS
- replication lag
- binary log 저장공간

느린 쿼리 로그 예:

```
[mysqld]
slow_query_log=ON
long_query_time=1
log_queries_not_using_indexes=OFF
```

log_queries_not_using_indexes 는 많은 쿼리를 기록할 수 있으므로 운영에서 무조건 켜지 않는다.

애플리케이션 로그에는 다음을 포함한다.

- timestamp
- log level
- request ID 또는 trace ID
- 요청 경로
- 상태 코드
- 처리시간

- 배포 버전
- 예외 스택

기록하면 안 되는 값:

- DB 비밀번호
- Authorization 헤더
- 세션 쿠키
- 주민등록번호
- 카드번호
- 전체 액세스 토큰
- MySQL 연결 문자열의 비밀번호

6. 운영 보안 체크리스트

Django:

- DEBUG=False
- 강력하고 비공개인 SECRET_KEY
- ALLOWED_HOSTS 제한
- HTTPS 강제
- secure cookie 적용
- CORS 와 CSRF 출처 제한
- 관리자 계정 MFA 또는 접근망 제한
- 오류 화면의 환경정보 노출 방지
- 사용자 업로드 검증
- 의존성 보안 업데이트

MySQL:

- 인터넷에 3306 포트 공개 금지
- VPC·방화벽·보안그룹으로 서버 접근 제한
- 애플리케이션 계정 최소 권한
- root 원격 접속 제한
- TLS 연결
- utf8mb4
- InnoDB 사용

- strict mode 적용
- 백업과 binary log 보존
- 감사 로그 및 접속 실패 감시
- 정기적인 비밀번호 교체
- 복제본과 백업 데이터도 암호화

최종 검사:

```
python manage.py check --deploy ₩
--settings=config.settings.production
```

핵심 정리

MySQL 기반 Django 실무 환경의 핵심은 다음과 같다.

1. django.db.backends.mysql 과 mysqlclient 를 사용한다.
2. 데이터베이스·테이블·연결 문자셋을 utf8mb4 로 통일한다.
3. 트랜잭션과 외래키가 필요한 모든 업무 테이블은 InnoDB 를 사용한다.
4. strict mode 로 잘못된 값의 자동 절단과 묵시적 변환을 방지한다.
5. 격리수준은 보통 READ COMMITTED 를 사용하되 업무 정합성에 맞춰 검토한다.
6. atomic()과 select_for_update()로 중요한 동시 갱신을 보호한다.
7. 교착상태는 완전 제거 대상이 아니라 감지·재시도·일관된 잠금 순서로 관리한다.
8. Gunicorn 워커 수와 Celery 프로세스 수를 기준으로 DB 연결 수를 계산한다.
9. 대형 테이블 DDL 은 Online DDL 여부와 메타데이터 락을 사전에 검증한다.
10. 백업은 mysqldump 만이 아니라 스냅샷, binary log, PITR, 복구 훈련까지 포함해야 한다.
11. 읽기 복제본은 복제 지연을 고려해 정합성이 중요한 조회와 분리한다.
12. 운영 배포는 마이그레이션, 롤링 배포, 헬스체크, 모니터링, 롤백 호환성까지 하나의 절차로 관리해야 한다.